Key Concepts, Models and Parameters

Product recommendation from transaction history: a reference built from first principles

November 2026

How to read this. Every concept used in the project is listed in the order you need it, starting from basic arithmetic on vectors and ending with how we judged the results. Each entry has four parts: *what it is*, *how it is used in this project*, a *derivation* from earlier entries where there is a formula, and a *worked example* with the project's own numbers where useful. Part I builds the mathematical toolkit; later parts use only what came before.

Contents

I Mathematical foundations **3**

1 Numbers, vectors and matrices **3**

2 Functions, derivatives and the chain rule **3**

3 Probability and statistics **4**

4 Correlated random traits: covariance and the Cholesky trick **4**

5 From odds to the sigmoid **4**

6 Likelihood and the binary cross-entropy loss **5**

7 Gradient descent and its variants **5**

7.1 Gradient descent . . . 5

7.2 Mini-batches and gradient accumulation . . . 5

7.3 Momentum, Adam and AdamW . . . 6

7.4 Learning-rate warm-up and cosine decay . . . 6

7.5 Gradient clipping . . . 6

8 Numbers inside a computer: 32-bit and 16-bit floats **6**

II Data **6**

9 Synthetic data and the hidden "true chance" **7**

10 Profile (tabular) features **7**

11 Time-based split and leakage **7**

III Turning transactions into model input **7**

12 Tokens and byte-pair encoding (BPE) **7**

13 Special transaction codes **8**

13.1 Amount ranges by quantiles 8
13.2 Context window and truncation 8

IV The small language model 8

14 Embeddings 8

15 Self-attention 8

16 Rotary position embeddings (RoPE) 9

17 RMSNorm, residual connections and SwiGLU 9

18 Qwen3-1.7B: the model and its parameter count 10

V Fine-tuning 10

19 Pre-training vs fine-tuning 10

20 LoRA: low-rank adaptation 10

21 Memory: why LoRA fits on a free GPU 11

22 Gradient checkpointing 11

VI The profile network and fusion 11

23 Neural-network building blocks 11

24 PLR numeric embeddings 11

25 DCNv2 cross network 12

26 Joint fusion and the per-product head 12

VII Baselines 12

27 Decision trees and gradient boosting (LightGBM) 12

28 Principal component analysis (PCA) 13

VIII Measuring results 13

29 AUC (area under the ROC curve) 13

30 PR-AUC (average precision) 13

31 Ranking metrics against the true chances 14

32 Uncertainty: standard error, bootstrap and test size **14**

IX Factors that affected the results **14**

33 Summary table **15**

Part I

Mathematical foundations

1 Numbers, vectors and matrices

What it is. A *vector* $\mathbf{x} \in \mathbb{R}^d$ is an ordered list of d real numbers. A *matrix* $W \in \mathbb{R}^{m \times n}$ is a grid of numbers with m rows and n columns. The *dot product* of two vectors of the same length is $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{d} a_i b_i$. Matrix–vector multiplication $W\mathbf{x}$ gives a vector whose i-th entry is the dot product of row i of W with $\mathbf{x}$.

In this project. Every model in the project is built from these. A customer's profile is a vector of 18 numbers; each transaction code becomes a vector of 2,048 numbers inside Qwen3; every "layer" is a matrix multiplication.

Linear layer. A *linear (fully connected) layer* maps $\mathbf{x} \in \mathbb{R}^n$ to $\mathbf{y} = W\mathbf{x} + \mathbf{b} \in \mathbb{R}^m$. It has mn weights plus m biases.

Worked example. The projection from Qwen3's 2,048-number summary to 128 numbers has $2048 \times 128 + 128 = 262{,}272$ parameters.

Element-wise product. $(\mathbf{a} \odot \mathbf{b})_i = a_i b_i$. Used in the DCNv2 cross layer (Section 25) and in SwiGLU (Section 17).

Norm. The length of a vector is $\|\mathbf{x}\| = \sqrt{\sum_i x_i^2}$. Used in gradient clipping and RMSNorm.

2 Functions, derivatives and the chain rule

What it is. The *derivative* $f'(x)$ is the slope of f at x: how much f changes per unit change in x, for very small changes,

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

For a function of many inputs, the *partial derivative* $\partial f / \partial x_i$ holds all other inputs fixed. The *gradient* ∇f collects all partial derivatives into a vector; it points in the direction of steepest increase.

Derivation. Three rules are enough for everything that follows.

1. *Power rule*: $(x^n)' = nx^{n-1}$. From the limit: $(x+h)^n = x^n + nx^{n-1}h + O(h^2)$, so the ratio tends to nx^{n-1}.
2. *Exponential*: $(e^x)' = e^x$, because $e^{x+h} - e^x = e^x(e^h - 1)$ and $(e^h - 1)/h \to 1$ by the definition of e.
3. *Logarithm*: $(\ln x)' = 1/x$, by differentiating $e^{\ln x} = x$: $e^{\ln x} \cdot (\ln x)' = 1$.

Chain rule. If $y = f(u)$ and $u = g(x)$, a small change Δx causes $\Delta u \approx g'(x)\Delta x$, which causes $\Delta y \approx f'(u)\Delta u$. Therefore

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

With many paths from x to y, the contributions add: $\dfrac{\partial y}{\partial x} = \sum_k \dfrac{\partial y}{\partial u_k} \dfrac{\partial u_k}{\partial x}$.

In this project. Training a network means computing the gradient of the loss with respect to millions of weights. *Backpropagation* is just the chain rule applied from the loss backwards through every layer, reusing intermediate results.

Gradient of a linear layer. For $\mathbf{y} = W\mathbf{x} + \mathbf{b}$ and a loss L with known $\mathbf{g} = \partial L / \partial \mathbf{y}$: since $y_i = \sum_j W_{ij} x_j + b_i$,

$$\frac{\partial L}{\partial W_{ij}} = g_i \, x_j \;\Rightarrow\; \frac{\partial L}{\partial W} = \mathbf{g}\mathbf{x}^\top, \qquad \frac{\partial L}{\partial \mathbf{b}} = \mathbf{g}, \qquad \frac{\partial L}{\partial \mathbf{x}} = W^\top \mathbf{g}.$$

The last result passes the gradient on to the previous layer. This one identity, applied repeatedly, is backpropagation.

3 Probability and statistics

What it is. A *random variable* takes values by chance. Its *expectation* $\mathbb{E}[X] = \sum_x x\,P(X = x)$ is the long-run average. Its *variance* $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2]$ measures spread; the *standard deviation* is $\sqrt{\mathrm{Var}}$.

Two facts used repeatedly.

1. Linearity: $\mathbb{E}[aX + bY] = a\mathbb{E}X + b\mathbb{E}Y$, directly from the definition of a sum.
2. For independent X, Y: $\mathrm{Var}(X+Y) = \mathrm{Var}\,X + \mathrm{Var}\,Y$, and $\mathrm{Var}(aX) = a^2\,\mathrm{Var}\,X$. *Proof:* expand $\mathbb{E}[(X - \mu_X + Y - \mu_Y)^2]$; the cross term $2\mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$ is zero when X, Y are independent.

Consequence: the average of n independent copies of X has variance $\mathrm{Var}(X)/n$, so its standard deviation shrinks as $1/\sqrt{n}$. This is why bigger test sets give tighter results (Section 32).

Distributions used in the data generator.

- *Bernoulli*(p): 1 with probability p, else 0. Mean p, variance $p(1-p)$. **Here:** whether a customer took the offered product.
- *Normal* $\mathcal{N}(\mu, \sigma^2)$: the bell curve, density $\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/2\sigma^2}$. **Here:** hidden traits, noise.
- *Log-normal*: e^Z with Z normal; always positive and skewed to the right. **Here:** income and transaction amounts.
- *Poisson*(λ): counts of rare events, $P(k) = e^{-\lambda}\lambda^k/k!$. **Here:** how many purchases of a category happen on a day.
- *Gamma*: positive and skewed. **Here:** how many days ago a life event happened, and customer tenure.

Poisson from first principles. Split a day into n tiny slots, each with a purchase with probability λ/n. The count is Binomial$(n, \lambda/n)$:

$$P(k) = \binom{n}{k}\left(\frac{\lambda}{n}\right)^k\left(1 - \frac{\lambda}{n}\right)^{n-k} \xrightarrow{n\to\infty} \frac{\lambda^k}{k!}e^{-\lambda},$$

because $\binom{n}{k}/n^k \to 1/k!$ and $(1 - \lambda/n)^n \to e^{-\lambda}$.

4 Correlated random traits: covariance and the Cholesky trick

What it is. The *covariance* of X and Y is $\mathbb{E}[(X - \mu_X)(Y - \mu_Y)]$; the *correlation* divides it by both standard deviations, giving a number in $[-1, 1]$. A *covariance matrix* Σ holds all pairwise covariances of a random vector.

In this project. Each synthetic customer has seven hidden traits (income, saver, traveller, stress, digital, discretionary, age). They should be related: richer people tend to save more, stressed people tend to save less. We specify a 7×7 correlation matrix R, for example corr(income, saver) $= 0.40$ and corr(saver, stress) $= -0.45$.

Derivation. We need random vectors $\mathbf{z}$ with $\mathrm{Cov}(\mathbf{z}) = R$. Start from $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ (independent standard normals) and set $\mathbf{z} = L\boldsymbol{\varepsilon}$. Then

$$\mathrm{Cov}(\mathbf{z}) = \mathbb{E}[L\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top L^\top] = L\,\mathbb{E}[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^\top]\,L^\top = LIL^\top = LL^\top.$$

So we need a matrix L with $LL^\top = R$. The *Cholesky decomposition* gives exactly that, with L lower-triangular; it exists only if R is positive definite, so the code also acts as a check that the chosen correlations are consistent.

5 From odds to the sigmoid

What it is. The *sigmoid* $\sigma(z) = \dfrac{1}{1 + e^{-z}}$ turns any real number into a probability between 0 and 1. The input z is called a *logit*.

Derivation. Probabilities live in $(0, 1)$, but linear models produce any real number. The *odds* $p/(1-p)$ live in $(0, \infty)$, and the *log-odds* $\ln\frac{p}{1-p}$ live in $(-\infty, \infty)$. Set the log-odds equal to a score z and solve for p:

$$\ln\frac{p}{1-p} = z \;\Rightarrow\; \frac{p}{1-p} = e^z \;\Rightarrow\; p = e^z - pe^z \;\Rightarrow\; p = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}.$$

Its derivative, by the chain rule: $\sigma'(z) = \dfrac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\big(1-\sigma(z)\big)$.

In this project. Every model outputs a logit for each product; the sigmoid turns it into the chance of take-up. The data generator also uses the sigmoid: each product's true chance is $\sigma(\text{trait-based logit} + \text{intercept})$.

Calibrating the intercept by bisection. The generator wants each product to be taken up by 8% of customers on average. The average of $\sigma(z_i + a)$ over customers rises steadily with the intercept a, so we can find the right a by *bisection*: start with $a \in [-10, 10]$, test the midpoint, keep the half that contains the target, and repeat 60 times. Each step halves the interval, so after 60 steps the error is below $20/2^{60} \approx 2 \times 10^{-17}$.

6 Likelihood and the binary cross-entropy loss

What it is. A *loss function* measures how wrong a prediction is; training minimises its average. *Binary cross-entropy* (BCE) is the loss for yes/no outcomes:

$$\ell(y, p) = -\big[y \ln p + (1-y)\ln(1-p)\big].$$

Derivation. *Maximum likelihood* says: choose parameters that make the observed outcomes most probable. For one customer with outcome $y \in \{0, 1\}$ and predicted chance p, the Bernoulli probability of what we saw is $p^y(1-p)^{1-y}$. For n independent customers the likelihood is the product $\prod_i p_i^{y_i}(1-p_i)^{1-y_i}$. Products of many small numbers are awkward, so take the logarithm (which keeps the same maximiser because ln is increasing), then flip the sign to get something to minimise:

$$-\ln \prod_i p_i^{y_i}(1-p_i)^{1-y_i} = \sum_i -\big[y_i \ln p_i + (1-y_i)\ln(1-p_i)\big] = \sum_i \ell(y_i, p_i).$$

Gradient with respect to the logit. With $p = \sigma(z)$ and $\sigma' = p(1-p)$:

$$\frac{\partial \ell}{\partial z} = -\Big[\frac{y}{p} - \frac{1-y}{1-p}\Big]p(1-p) = -\big[y(1-p) - (1-y)p\big] = p - y.$$

This is remarkably simple: the error signal is just "predicted minus actual".

In this project. Both fine-tuned models are trained with BCE on the offered product (PyTorch's `binary_cross_entropy_with_logits`, which combines the sigmoid and the loss in one numerically safe step).

Worked example. A customer who took the product ($y = 1$) with predicted $p = 0.089$ has loss $-\ln 0.089 = 2.42$ and gradient $0.089 - 1 = -0.911$, pushing the logit up strongly. One who did not ($y = 0$) with $p = 0.089$ has loss $-\ln 0.911 = 0.093$ and gradient $+0.089$, a gentle push down.

7 Gradient descent and its variants

7.1 Gradient descent

What it is. To minimise a loss $L(\theta)$, repeatedly step against the gradient: $\theta \leftarrow \theta - \eta \nabla L(\theta)$, where η is the *learning rate*.

Derivation. By a first-order Taylor expansion, $L(\theta + \Delta) \approx L(\theta) + \nabla L^\top \Delta$. Among steps of fixed length, $\nabla L^\top \Delta$ is most negative when Δ points opposite to ∇L (the dot product is smallest when vectors point in opposite directions). Hence $\Delta = -\eta \nabla L$.

7.2 Mini-batches and gradient accumulation

What it is. The true gradient is an average over all training customers. A *mini-batch* estimates it from a few customers. *Gradient accumulation* adds up gradients from several small batches before taking one step.

Derivation. Since the gradient of an average is the average of gradients, $\nabla \frac{1}{32}\sum_{i=1}^{32} \ell_i = \frac{1}{8}\sum_{k=1}^{8} \nabla \frac{1}{4}\sum_{i \in B_k} \ell_i$. Dividing each small batch's loss by 8 and summing the gradients gives exactly the gradient of a batch of 32.

In this project. Only 4 customers fit on a T4 at once, so we accumulate 8 batches: an effective batch of 32. With 13,265 training customers, one pass is $\lceil 13265/32 \rceil = 415$ planned updates; 414 were completed.

7.3 Momentum, Adam and AdamW

What it is. *Adam* keeps running averages of the gradient and of its square, and scales each weight's step by them. *AdamW* adds weight decay separately from the gradient.

Derivation. Let g_t be the gradient at step t.

1. First moment (momentum), an exponential moving average: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$, with $\beta_1 = 0.9$. It smooths noisy gradients.
2. Second moment: $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$, with $\beta_2 = 0.999$. It estimates each weight's typical gradient size.
3. Bias correction. With $m_0 = 0$, unrolling gives $m_t = (1-\beta_1)\sum_{k=1}^{t} \beta_1^{t-k} g_k$. If gradients have constant mean $\bar{g}$, then $\mathbb{E}[m_t] = \bar{g}(1-\beta_1)\sum_{j=0}^{t-1}\beta_1^j = \bar{g}(1-\beta_1^t)$, using the geometric series $\sum_{j=0}^{t-1}\beta^j = \frac{1-\beta^t}{1-\beta}$. So divide by $(1-\beta_1^t)$: $\hat{m}_t = m_t/(1-\beta_1^t)$, and likewise $\hat{v}_t = v_t/(1-\beta_2^t)$.
4. Update: $\theta \leftarrow \theta - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t}+\epsilon)$. Dividing by $\sqrt{\hat{v}_t}$ makes the step roughly η in size for every weight, whatever its gradient scale.
5. AdamW: also shrink weights directly, $\theta \leftarrow \theta - \eta\lambda\theta$. Keeping this outside the $\hat{m}/\sqrt{\hat{v}}$ scaling makes the decay act evenly on all weights.

In this project. AdamW with learning rate 2×10^{-4} for LoRA weights and 10^{-3} for the heads; weight decay 10^{-4} on the profile network only.

Worked example. At step 1, $m_1 = 0.1 g_1$ and $\hat{m}_1 = 0.1 g_1/0.1 = g_1$: bias correction removes the start-from-zero shrinkage exactly.

7.4 Learning-rate warm-up and cosine decay

What it is. The learning rate is ramped up linearly for the first steps, then lowered along half a cosine wave to zero.

Derivation. With S total steps and w warm-up steps, the multiplier is

$$f(s) = \begin{cases} s/w, & s < w \\ \frac{1}{2}\Big(1 + \cos\big(\pi \frac{s-w}{S-w}\big)\Big), & s \geq w. \end{cases}$$

At $s = w$ the cosine is $\cos 0 = 1$ so $f = 1$; at $s = S$ it is $\cos \pi = -1$ so $f = 0$. Warm-up avoids large early steps while Adam's averages are still unreliable; the slow cosine end lets the model settle.

Worked example. $S = 414$, $w = 20$ (5%). LoRA learning rate: step 10: 1.0×10^{-4}; step 20: 2.0×10^{-4}; step 217 (halfway through decay): 1.0×10^{-4}; step 300: 3.9×10^{-5}.

7.5 Gradient clipping

What it is. If the gradient vector is longer than a threshold c, scale it down to length c: $g \leftarrow g \cdot \min(1, c/\|g\|)$.

In this project. $c = 1.0$. It stops a single unusual batch from making a damaging step.

8 Numbers inside a computer: 32-bit and 16-bit floats

What it is. A floating-point number stores a sign, an exponent and a mantissa: $x = \pm 1.m \times 2^e$. *fp32* uses 32 bits (8 exponent, 23 mantissa); *fp16* uses 16 bits (5 exponent, 10 mantissa).

Derivation. fp16's largest value is $(2 - 2^{-10}) \times 2^{15} = 65{,}504$, and its smallest normal positive value is $2^{-14} \approx 6.1 \times 10^{-5}$ (down to 6×10^{-8} with reduced precision). With 10 mantissa bits, relative precision is about $2^{-11} \approx 0.05\%$. Small gradients, such as 10^{-8}, round to zero (*underflow*).

Loss scaling. Multiply the loss by a large factor s (for example 2^{16}) before backpropagation. By the chain rule every gradient is multiplied by s, lifting small values above the underflow limit. Divide by s before the optimiser step. If any gradient overflows to infinity, skip the step and halve s. `GradScaler` does this automatically.

In this project. The frozen Qwen3 weights are stored in fp16 (2 bytes each); the trainable LoRA weights in fp32 for precise updates; matrix maths runs in fp16 inside `autocast`. The T4 GPU is fastest in fp16 and does not support bf16 efficiently.

Part II

Data

9 Synthetic data and the hidden "true chance"

What it is. Synthetic data is generated by a program instead of collected from real people. Because we write the rules, we know each customer's true probability of taking each product.

In this project. 20,000 customers, 150 days of history each, 4,251,789 transactions, 5 products. The generator:

1. samples seven correlated hidden traits per customer (Section 4);
2. gives 60% of customers one life event (pay rise, financial stress, new car, baby), usually recent;
3. simulates salary and bills on fixed days, everyday purchases as Poisson counts with log-normal amounts, and bursts for trips and stress;
4. computes each product's logit from traits and recent events, adds noise ($\sigma = 0.5$), and calibrates the intercept so the average chance is 8%;
5. offers one product per customer (70% random, 30% targeted at the highest noisy logit), and draws the outcome from Bernoulli(true chance).

Why the noise matters. Even a model that knew every hidden trait could not predict perfectly, because the outcome is a coin flip with the true chance. That ceiling is the "highest possible" AUC of 0.867.

Targeted offers. When offers are targeted, customers who would take the product anyway are offered it more often. This *selection bias* mirrors real campaigns, and is why the observed take-up rate (12.9%) is above the 8% base rate.

10 Profile (tabular) features

What it is. Tabular features are one row of numbers per customer.

In this project. 18 numbers (age, tenure, declared income, credit limit, app sessions, children declared, 90-day transaction count, inflow, outflow, spend in 8 broad categories, overdraft flag) and 2 categories (region, channel). They are deliberately lossy: income is noisy, categories are broad, and everything is summed over 90 days.

Log transform. Money is skewed (a few very large values). Applying $\text{sign}(x)\ln(1 + |x|)$ compresses large values: R$100 becomes 4.6 and R$10,000 becomes 9.2.

Standardisation (z-score). $x' = (x - \mu)/s$, where μ, s are the mean and standard deviation on training customers only. Every feature then has mean 0 and spread 1, which keeps gradient steps balanced across features.

11 Time-based split and leakage

What it is. *Leakage* is when information from the future, or from the test set, influences training, making results look better than they will be in real use.

In this project. Customers scored in months 1–8 train the model (13,265), month 9 tunes it (1,666), months 10–12 test it (5,069). Amount ranges and feature scaling are fitted on training customers only, and every transaction is dated before the customer's scoring date.

Part III

Turning transactions into model input

12 Tokens and byte-pair encoding (BPE)

What it is. A language model reads *tokens*: pieces of text from a fixed vocabulary. *Byte-pair encoding* builds that vocabulary by starting from single characters (bytes) and repeatedly merging the most frequent adjacent

pair into a new token, until the vocabulary reaches a target size.

In this project. Qwen3's BPE vocabulary has 151,936 tokens. Merchant names are split by it: "NETFLIX.COM" becomes `NET`, `FL`, `IX`, `.COM`. We keep the first 6 pieces of each name.

13 Special transaction codes

What it is. We add 74 codes of our own that describe each transaction's structure.

In this project. Each transaction becomes `<TX> <SIGN> <AMT_b> <MON_m> <DAY_d> <WD_w>` followed by up to 6 merchant-name pieces: 1 start code, 2 sign codes (money in or out), 21 amount ranges, 12 months, 31 days, 7 weekdays.

13.1 Amount ranges by quantiles

What it is. A *quantile* splits sorted values into equal-sized groups. With 21 ranges we need 20 cut points: the $1/21$, $2/21$, ..., $20/21$ quantiles.

Derivation. Amounts are first compressed with $a \mapsto \ln(1 + |a|)$, so that ranges are about equally wide in relative terms (R\$10 vs R\$20 matters as much as R\$1,000 vs R\$2,000). The range number is how many cut points lie at or below the value: $b = \#\{k : e_k \leq \ln(1 + |a|)\}$. Equal-frequency ranges mean each code is used about equally often, so each one gets enough training examples.

Worked example. A payment of R\$48.90: $\ln(49.90) = 3.910$. The cut points begin 2.345, 2.809, 3.078, 3.325, 3.499, 3.675, 3.799, 3.955, ...; seven are below 3.910, so the code is `<AMT_7>`, which covers about R\$43.66 to R\$51.20.

13.2 Context window and truncation

What it is. The *context window* is the maximum number of tokens the model reads at once.

In this project. 1,024 tokens in round 2 (512 in round 1). On average a transaction uses 11.7 tokens, so the window holds $1024/11.7 \approx 87$ of the most recent transactions, about 9 weeks of a median customer's 150 days. We drop the oldest transactions (left truncation) because recent behaviour predicts best, and pad on the left so the newest token is always in the last position.

Part IV

The small language model

14 Embeddings

What it is. An *embedding* table stores one learned vector per token. Looking up token t returns row t.

Derivation. A lookup equals multiplying a one-hot vector $\mathbf{e}_t$ (all zeros except a 1 at position t) by the table E: $\mathbf{e}_t^\top E = E_{t,:}$. So it is a linear layer that skips the multiplications by zero, and its gradient only touches the row that was used.

In this project. Qwen3's table is $151{,}936 \times 2{,}048 = 311$ million numbers. Adding our 74 codes to it would make the whole table trainable, so we keep a separate $74 \times 2{,}048$ table (151,552 numbers). Each code starts as the average embedding of a matching phrase; for example `<MON_11>` starts from the vector for "november" and `<SIGN_OUT>` from "debit paid".

15 Self-attention

What it is. Attention lets each position in a sequence gather information from other positions, weighting them by relevance.

Derivation. For each position i with input vector $\mathbf{x}_i$, three linear maps produce a *query* $\mathbf{q}_i = W_Q \mathbf{x}_i$, a *key* $\mathbf{k}_i = W_K \mathbf{x}_i$ and a *value* $\mathbf{v}_i = W_V \mathbf{x}_i$, each of length d_k.

1. Relevance of j to i: the dot product $s_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ is large when the vectors point the same way.
2. Turn scores into weights that are positive and sum to 1, using *softmax*: $a_{ij} = e^{s_{ij}} / \sum_{j'} e^{s_{ij'}}$. (The exponential makes everything positive and preserves order; dividing by the sum normalises.)
3. Output: the weighted average of values, $\mathbf{o}_i = \sum_j a_{ij} \mathbf{v}_j$.

In matrix form: $\text{Attention}(Q, K, V) = \text{softmax}\big(QK^\top / \sqrt{d_k}\big)V$.

Why divide by $\sqrt{d_k}$. Suppose the components of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean 0 and variance 1. Each product $q_i k_i$ then has mean 0 and variance $\mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$. The dot product is a sum of d_k such terms, so by Section 3 its variance is d_k and its standard deviation $\sqrt{d_k}$. With $d_k = 128$ the scores would typically be ± 11.3; softmax of such large numbers puts almost all weight on one position, and its gradients become nearly zero. Dividing by $\sqrt{d_k}$ brings the variance back to 1.

Causal mask. Qwen3 predicts text left to right, so position i may only look at positions $j \le i$: scores for $j > i$ are set to $-\infty$, giving weight $e^{-\infty} = 0$. Therefore only the *last* position has seen every transaction, which is why we take its output as the customer summary (*last-token pooling*).

Multi-head and grouped-query attention. Instead of one attention with $d_k = 2048$, Qwen3 runs 16 heads of size 128 in parallel and joins their outputs; each head can focus on a different pattern. In *grouped-query attention* (GQA), the 16 query heads share 8 key/value heads (two queries per key/value pair), which halves the size of W_K, W_V with little loss of quality.

16 Rotary position embeddings (RoPE)

What it is. Attention by itself ignores order. RoPE encodes each token's position by rotating its query and key vectors by an angle that grows with position.

Derivation. Group the vector into pairs (x_{2i}, x_{2i+1}) and treat each pair as a point in the plane. At position m, rotate pair i by angle $m\theta_i$, with $\theta_i = b^{-2i/d}$ (Qwen3 uses $b = 10^6$):

$$R(m\theta) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}.$$

Rotation matrices satisfy $R(\alpha)^\top R(\beta) = R(\beta - \alpha)$ (rotating back by α then forward by β). Hence for a query at position m and a key at position n,

$$(R(m\theta)\mathbf{q})^\top (R(n\theta)\mathbf{k}) = \mathbf{q}^\top R(m\theta)^\top R(n\theta)\mathbf{k} = \mathbf{q}^\top R\big((n-m)\theta\big)\mathbf{k}.$$

The score depends only on the *distance* $n - m$, not on absolute positions. Different pairs rotate at different speeds, like the hands of a clock, so both short and long distances are represented.

In this project. Inherited from Qwen3. It lets the model tell "salary yesterday" from "salary 60 transactions ago".

17 RMSNorm, residual connections and SwiGLU

RMSNorm. $\text{RMSNorm}(\mathbf{x}) = \dfrac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}$. It rescales each vector to a typical size, so values neither explode nor vanish as they pass through 28 layers. It is a simpler form of LayerNorm (which also subtracts the mean). Qwen3 also normalises queries and keys before attention (*QK-norm*).

Residual connection. Each block computes $\mathbf{x} \leftarrow \mathbf{x} + F(\mathbf{x})$. By the chain rule $\partial(\mathbf{x} + F(\mathbf{x}))/\partial\mathbf{x} = I + \partial F/\partial\mathbf{x}$; the identity term I lets gradients flow straight back through all 28 layers.

SwiGLU feed-forward block. After attention, each position passes through

$$\text{FFN}(\mathbf{x}) = W_{\text{down}}\big(\,\text{SiLU}(W_{\text{gate}}\mathbf{x}) \odot W_{\text{up}}\mathbf{x}\big), \quad \text{SiLU}(z) = z\,\sigma(z).$$

$W_{\text{gate}}, W_{\text{up}}$ map 2,048 to 6,144 numbers and W_{down} maps back. The gate decides, number by number, how much of the "up" signal to pass.

18 Qwen3-1.7B: the model and its parameter count

What it is. A *small language model* (SLM) is a language model small enough to run and fine-tune on modest hardware, here about 1.7 billion parameters.

In this project. Qwen3-1.7B: 28 layers, hidden size $d = 2048$, 16 query heads and 8 key/value heads of size 128, feed-forward size 6,144, vocabulary 151,936. We remove its word-prediction layer and use the last hidden state.

Derivation. Count the weights per layer from the shapes:

$$\begin{aligned}
\text{attention:} &\quad \underbrace{2048 \cdot 2048}_{W_Q} + \underbrace{2 \cdot 2048 \cdot 1024}_{W_K, W_V} + \underbrace{2048 \cdot 2048}_{W_O} = 12{,}582{,}912 \\
\text{feed-forward:} &\quad 3 \cdot 2048 \cdot 6144 = 37{,}748{,}736 \\
\text{norms:} &\quad 2 \cdot 2048 + 2 \cdot 128 = 4{,}352 \\
\text{per layer:} &\quad 50{,}336{,}000; \quad \times 28 = 1{,}409{,}408{,}000 \\
\text{embeddings:} &\quad 151{,}936 \cdot 2048 = 311{,}164{,}928
\end{aligned}$$

Total ≈ 1.72 billion (the word-prediction layer shares the embedding weights, so it adds nothing).

Compute per token. A forward pass costs about 2 operations per weight (one multiply, one add), so $2N$. Training normally costs $6N$ per token ($2N$ forward, $4N$ backward). With frozen weights the backward pass still needs gradients with respect to activations ($2N$) but almost none for weights; gradient checkpointing adds a second forward ($2N$). Either way, about $6N \approx 10$ billion operations per token.

Worked example. One training pass reads $13{,}265\times \sim 1{,}019 \approx 13.5$ million tokens: $6 \times 1.72 \times 10^9 \times 1.35 \times 10^7 \approx 1.4 \times 10^{17}$ operations. A T4's fp16 peak is 65 trillion per second, so the ideal time is 36 minutes. The real run took 4.1 hours, about 15% of peak, which is typical for small batches with checkpointing.

Part V

Fine-tuning

19 Pre-training vs fine-tuning

What it is. *Pre-training* teaches a model general patterns from huge unlabelled data (Qwen3: web text, predicting the next word). *Fine-tuning* adapts it to a specific task with labelled examples.

In this project. We fine-tuned on 13,265 labelled customers. We did not pre-train on transactions; doing so (predict the next transaction across all 4.25 million) is the top suggested improvement.

20 LoRA: low-rank adaptation

What it is. Instead of changing a large weight matrix $W_0 \in \mathbb{R}^{m \times n}$, LoRA freezes it and learns a small correction $\Delta W = BA$ with $B \in \mathbb{R}^{m \times r}$, $A \in \mathbb{R}^{r \times n}$ and small rank r:

$$\mathbf{y} = W_0 \mathbf{x} + \frac{\alpha}{r} BA\mathbf{x}.$$

Derivation.

1. *Rank.* The rank of a matrix is the number of independent directions it can produce. BA has rank at most r, because every output $BA\mathbf{x} = B(A\mathbf{x})$ is a combination of the r columns of B. The bet is that task adaptation needs only a few directions.
2. *Parameter saving.* Full update: mn numbers. LoRA: $r(m+n)$. For a 2048×2048 matrix with $r = 16$: 65,536 instead of 4,194,304, 64 times fewer.
3. *Starting point.* A is random and $B = 0$, so $\Delta W = 0$ at the start and the model begins exactly as the original.
4. *Gradients.* Write $\mathbf{h} = A\mathbf{x}$ and $\mathbf{g} = \partial L / \partial \mathbf{y}$. By the linear-layer rule (Section 2): $\partial L / \partial B = \frac{\alpha}{r} \mathbf{g}\mathbf{h}^\top$ and $\partial L / \partial A = \frac{\alpha}{r} B^\top \mathbf{g}\mathbf{x}^\top$. At the start $B = 0$, so only B moves; once B is non-zero, A starts moving too.
5. *Scale α/r.* It keeps the size of the update stable when you change r, so the learning rate need not be retuned. Here $\alpha/r = 32/16 = 2$.

In this project. $r = 16$, $\alpha = 32$, dropout 0.05, applied to all seven projections in all 28 layers.

Worked example. Per layer: q: $16(2048 + 2048) = 65{,}536$; k and v: $16(2048 + 1024) = 49{,}152$ each; o: 65,536; gate and up: $16(2048 + 6144) = 131{,}072$ each; down: 131,072. Sum 622,592; $\times 28 = 17{,}432{,}576$. Adding the special-code table (151,552) and the heads gives 17.9M (transactions-only) and 18.3M (joint) trainable parameters, about 1% of the model.

21 Memory: why LoRA fits on a free GPU

Derivation. Full fine-tuning with AdamW in fp32 stores, per weight: the weight (4 bytes), its gradient (4), and Adam's m and v (4 each): 16 bytes. For 1.72B weights: 27.5 GB, more than a T4's 16 GB. With LoRA: frozen weights in fp16 ($1.72 \times 10^9 \times 2 = 3.4$ GB) plus 18M trainable weights at 16 bytes (0.3 GB): about 3.7 GB. The rest of the measured 7.0 GB peak is activations.

22 Gradient checkpointing

What it is. Backpropagation needs every layer's intermediate results (activations). *Checkpointing* stores only each layer's input and recomputes the rest during the backward pass.

Derivation. Storing all activations costs memory proportional to (layers $\times$ activation size per layer). Storing only the 28 layer inputs reduces this to roughly one layer's worth at a time plus the inputs, at the price of one extra forward pass (about 33% more compute, since forward is $2N$ of the $6N$).

In this project. Enabled for Qwen3; it is what makes a 1,024-token window fit in 7 GB.

Part VI
The profile network and fusion

23 Neural-network building blocks

ReLU. $\mathrm{ReLU}(z) = \max(0, z)$. Without a non-linear function between layers, stacked linear layers collapse into one linear layer ($W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$), so non-linearity is what lets networks model curves and interactions. Its derivative is 1 for $z > 0$ and 0 otherwise.

LayerNorm. $\mathrm{LN}(\mathbf{x}) = \gamma \odot \dfrac{\mathbf{x} - \mu}{\sqrt{s^2 + \epsilon}} + \beta$, with μ, s^2 the mean and variance of the vector's own entries. Applied to Qwen3's summary before the 2,048-to-128 projection, so the head sees inputs of stable size.

Dropout. During training, set each value to 0 with probability p and multiply survivors by $1/(1-p)$. The expected value is unchanged: $\mathbb{E}[\text{out}] = (1-p) \cdot \frac{x}{1-p} + p \cdot 0 = x$. It stops the network relying on any single value. Here $p = 0.1$ in the heads and 0.05 in LoRA.

24 PLR numeric embeddings

What it is. *Periodic–Linear–ReLU* embeddings turn each single number into a richer vector before the network sees it.

Derivation. For feature value x, learn $k = 16$ frequencies $c_1, \ldots, c_k$ and compute

$$\mathbf{v}(x) = \big[\sin(2\pi c_1 x), \cos(2\pi c_1 x), \ldots, \sin(2\pi c_k x), \cos(2\pi c_k x)\big] \in \mathbb{R}^{32},$$

then a linear layer to 16 numbers and a ReLU. A plain network with one input number tends to learn only smooth, slowly-varying functions of it. A combination of sines and cosines at several frequencies can approximate sharp or wiggly functions (this is the idea of a Fourier series), so thresholds such as "income above R$5,000" become easy to express.

In this project. Applied to each of the 18 profile numbers: $18 \times 16 = 288$ numbers, plus two 16-number embeddings for region and channel: 320 in total.

25 DCNv2 cross network

What it is. A *cross layer* explicitly multiplies features together, so the network can learn interactions such as income × region.

Derivation. With $\mathbf{x}_0$ the input and $\mathbf{x}_l$ the output of layer l:

$$\mathbf{x}_{l+1} = \mathbf{x}_0 \odot (W_l \mathbf{x}_l + \mathbf{b}_l) + \mathbf{x}_l.$$

Each entry of $W_l \mathbf{x}_l$ is a weighted sum of entries of $\mathbf{x}_l$; multiplying element-wise by $\mathbf{x}_0$ creates products $x_{0,i}\, x_{l,j}$. If $\mathbf{x}_l$ contains terms up to degree $l+1$ in the inputs, then $\mathbf{x}_{l+1}$ contains terms up to degree $l+2$. Starting from degree 1, three cross layers give feature interactions up to degree 4. The $+\mathbf{x}_l$ is a residual connection that keeps the lower-degree terms.

In this project. Three cross layers on the 320-number input, then a small network ($320 \to 256 \to 64$) giving a 64-number profile summary.

26 Joint fusion and the per-product head

What it is. *Fusion* combines the transaction summary and the profile summary. In *late fusion*, the language model is trained first and its output is handed to a separate model. In *joint fusion*, all parts are trained together on one loss.

Derivation. Let $\mathbf{h}$ be Qwen3's summary, $\mathbf{u} = \text{proj}(\text{LN}(\mathbf{h})) \in \mathbb{R}^{128}$, $\mathbf{t} = \text{DCN}(\text{profile}) \in \mathbb{R}^{64}$, and logits $\mathbf{z} = \text{MLP}([\mathbf{u}; \mathbf{t}]) \in \mathbb{R}^5$. For a customer offered product k, the loss is $\ell(y, \sigma(z_k))$. By Section 6 and the chain rule,

$$\frac{\partial \ell}{\partial \theta_{\text{LoRA}}} = (p_k - y)\, \frac{\partial z_k}{\partial \mathbf{u}}\, \frac{\partial \mathbf{u}}{\partial \mathbf{h}}\, \frac{\partial \mathbf{h}}{\partial \theta_{\text{LoRA}}}.$$

The error $p_k - y$ is the error of the *combined* prediction. If the profile already explains a customer well, $p_k - y$ is small and the language model receives little signal; where the profile is wrong, the language model is pushed to supply what is missing. In late fusion the language model is trained against its own error instead, so it may relearn what the profile already knows.

Per-product head. The last layer has 5 outputs, one per product. Only the offered product's output enters the loss (selected with `gather`), because we observe the outcome only for that product. At prediction time all 5 are available, and the top one is offered.

In this project. Joint model: $192 \to 128 \to 5$. Transactions-only model: $128 \to 128 \to 5$, without the profile network.

Part VII

Baselines

27 Decision trees and gradient boosting (LightGBM)

What it is. A *decision tree* splits customers by questions ("income > 5,000?") and gives each final group (leaf) a score. *Gradient boosting* adds trees one at a time, each correcting the errors of those before it. *LightGBM* is a fast implementation.

Derivation. Let $F_{m-1}(x)$ be the current total logit and add a tree f: $F_m = F_{m-1} + \eta f$. Approximate each customer's loss by a second-order Taylor expansion around F_{m-1}:

$$\ell(y_i, F_{m-1} + f) \approx \ell_i + g_i f(x_i) + \tfrac{1}{2} h_i f(x_i)^2,$$

where, from Section 6, $g_i = \partial \ell / \partial F = p_i - y_i$ and $h_i = \partial^2 \ell / \partial F^2 = p_i(1 - p_i)$. A tree gives the same value w to every customer in a leaf. Summing over leaf j with $G_j = \sum g_i$, $H_j = \sum h_i$, and adding a penalty $\frac{1}{2}\lambda w^2$:

$$\text{loss}_j(w) = G_j w + \tfrac{1}{2}(H_j + \lambda) w^2.$$

Setting the derivative to zero: $G_j + (H_j + \lambda)w = 0$, so the best leaf value is $w_j^* = -\dfrac{G_j}{H_j + \lambda}$, and the loss there

is $-\frac{1}{2}\frac{G_j^2}{H_j+\lambda}$. A split of a leaf into left and right parts is worth

$$\text{Gain} = \frac{1}{2}\Big[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\Big].$$

LightGBM tries every feature and threshold (using histograms to make this fast) and grows the tree at the leaf with the largest gain.

In this project. Up to 3,000 trees, learning rate 0.03, 31 leaves, stopping early when the validation score stops improving for 150 trees; it stopped at 293 trees. This is the standard model, AUC 0.765.

28 Principal component analysis (PCA)

What it is. PCA finds the few directions along which a set of vectors varies most, and describes each vector by its coordinates along them.

Derivation. For centred data with covariance Σ, the variance along a unit direction $\mathbf{w}$ is $\mathbf{w}^\top\Sigma\mathbf{w}$. Maximise it subject to $\mathbf{w}^\top\mathbf{w} = 1$ using a Lagrange multiplier: setting the derivative of $\mathbf{w}^\top\Sigma\mathbf{w} - \lambda(\mathbf{w}^\top\mathbf{w} - 1)$ to zero gives $\Sigma\mathbf{w} = \lambda\mathbf{w}$. So the best directions are eigenvectors of Σ, and the variance captured is the eigenvalue λ. Keep the 32 with the largest eigenvalues.

In this project. The two-step (late fusion) model reduces Qwen3's 2,048-number summary to 32 numbers with PCA, then adds them to LightGBM's inputs.

Part VIII

Measuring results

29 AUC (area under the ROC curve)

What it is. AUC is the probability that a randomly chosen customer who took the product is scored higher than a randomly chosen one who did not. 0.5 is random guessing; 1.0 is perfect.

Derivation. For a threshold t, the *true positive rate* is $\text{TPR}(t) = P(s > t \mid y = 1)$ and the *false positive rate* is $\text{FPR}(t) = P(s > t \mid y = 0)$. The ROC curve plots TPR against FPR as t falls from $+\infty$ to $-\infty$. Its area is

$$\text{AUC} = \int_0^1 \text{TPR}\, d(\text{FPR}).$$

Let f_0 be the score density of negatives. As t decreases by dt, FPR increases by $f_0(t)\,dt$, so

$$\text{AUC} = \int P(s_+ > t)\, f_0(t)\, dt = P(s_+ > s_-),$$

the probability that a positive outscores a negative. On data, this is the fraction of (positive, negative) pairs ranked correctly (ties count as half), which is the Mann–Whitney statistic.

In this project. The main score for every model. AUC only depends on the ranking, so it is unaffected by whether the probabilities are well calibrated.

Worked example. Positives scored 0.9, 0.6, 0.4; negatives 0.7, 0.3, 0.2. Of 9 pairs, 7 are ordered correctly (only 0.6 and 0.4 lose to 0.7), so AUC $= 7/9 = 0.778$.

Relative lift. $(\text{AUC}_{\text{new}} - \text{AUC}_{\text{base}})/\text{AUC}_{\text{base}}$. Ours: $(0.7362 - 0.7653)/0.7653 = -3.8\%$.

30 PR-AUC (average precision)

What it is. *Precision* is the share of customers flagged as likely who really took the product; *recall* is the share of all takers that were flagged. PR-AUC averages precision over all recall levels.

In this project. With only 12.9% takers, PR-AUC focuses on the top of the ranking, where offers are made. A random model scores 0.129. LightGBM: 0.372; our model: 0.332.

31 Ranking metrics against the true chances

Because the generator knows each customer's true chance for all 5 products, we can check whether a model orders the products correctly.

Hit@1. Share of customers for whom the model's top product is the truly best product. Random: $1/5 = 20\%$. LightGBM 56%, our model 44%.

NDCG@3 (normalised discounted cumulative gain). Score the top 3 recommended products by their true chances, discounting lower positions:

$$\mathrm{DCG@3} = \sum_{i=1}^{3} \frac{\text{true chance of the } i\text{-th recommended product}}{\log_2(i+1)}, \qquad \mathrm{NDCG@3} = \frac{\mathrm{DCG@3}}{\text{DCG@3 of the perfect order}}.$$

The logarithm discount reflects that lower slots get less attention; dividing by the perfect order puts the score between 0 and 1.

Worked example. True chances: investments 0.089, insurance 0.019, travel 0.012, cashback 0.009, loan 0.006. A model recommends travel, investments, insurance. $\mathrm{DCG} = 0.012/1 + 0.089/1.585 + 0.019/2 = 0.0777$. Perfect $\mathrm{DCG} = 0.089 + 0.019/1.585 + 0.012/2 = 0.1070$. $\mathrm{NDCG@3} = 0.726$.

Spearman rank correlation. The Pearson correlation of the two rankings. With no ties and rank differences d_i over n items,

$$\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}.$$

Derivation: ranks $1..n$ have mean $\frac{n+1}{2}$ and variance $\frac{n^2-1}{12}$. Using $\sum d_i^2 = \sum (a_i - b_i)^2 = \sum a_i^2 + \sum b_i^2 - 2\sum a_i b_i$, the covariance of the ranks equals $\frac{n^2-1}{12} - \frac{1}{2n}\sum d_i^2$; dividing by the variance gives the formula.

Worked example. True ranks (1,2,3,4,5), predicted (2,1,3,5,4): $\sum d^2 = 4$, $\rho = 1 - 24/120 = 0.8$. In round 1 the transactions-only model averaged -0.22 (worse than random); in round 2, $+0.22$.

32 Uncertainty: standard error, bootstrap and test size

What it is. A score measured on a sample of customers would come out a bit different on another sample. The *standard error* (SE) is the typical size of that difference; a 95% interval is about ± 1.96 SE.

Derivation. *Hanley–McNeil formula.* With n_1 positives, n_2 negatives and $Q_1 = \frac{A}{2-A}$, $Q_2 = \frac{2A^2}{1+A}$:

$$\mathrm{SE}(A) = \sqrt{\frac{A(1-A) + (n_1-1)(Q_1 - A^2) + (n_2-1)(Q_2 - A^2)}{n_1 n_2}}.$$

It follows from writing AUC as an average over pairs and computing the variance of that average, where Q_1 and Q_2 are the probabilities that two positives both outscore one negative, and that one positive outscores two negatives.

Worked example. Our test set: 653 takers and 4,416 non-takers. For $A = 0.736$: $\mathrm{SE} = 0.0116$, so the 95% interval is ± 0.023.

Paired bootstrap. Resample the 5,069 test customers with replacement 2,000 times; on each resample compute the difference between two models' AUCs on the *same* customers. The middle 95% of those differences is the interval. Pairing removes customer-to-customer variation shared by both models.

Worked example. Round 1 vs round 2, transactions-only: $+0.045$ (95% range $+0.029$ to $+0.062$), a real improvement. Joint model: $+0.0005$ (-0.009 to $+0.010$), no change.

Test size needed to detect a small gain. To detect a difference Δ with 95% confidence and 80% power, we need $\Delta \geq (1.96 + 0.84)\,\mathrm{SE}_{\mathrm{diff}}$. For two models with correlation ρ between their scores, $\mathrm{SE}_{\mathrm{diff}} = \mathrm{SE}\sqrt{2(1-\rho)}$, and SE shrinks as $1/\sqrt{n}$.

Worked example. A 1.25% gain at AUC 0.765 is $\Delta = 0.0096$, so we need $\mathrm{SE}_{\mathrm{diff}} \leq 0.0096/2.8 = 0.0034$. Today $\mathrm{SE} = 0.0113$ with $n = 5{,}069$. Assuming $\rho = 0.5$, $\mathrm{SE}_{\mathrm{diff}} = \mathrm{SE}$, so $n \approx 5{,}069 \times (0.0113/0.0034)^2 \approx 55{,}000$ customers.

Part IX

Factors that affected the results

33 Summary table

Factor	Value used	Effect on this project
Training customers	13,265	The largest limit. Fine-tuning 18M parameters from 13k examples gives a weak signal; tree models like LightGBM usually need much less data to do well.
Transaction pre-training	None	Qwen3 knows text, not spending patterns. Teaching those first is the top suggested improvement.
Context window	512 → 1,024 tokens	Doubling it raised transactions-only AUC from 0.623 to 0.668 (together with the head change).
Codes per transaction	13.7 → 11.7	Capping merchant names at 6 pieces fits more history in the same window.
Output head	offer as input → one output per product	Product ranking went from worse than random to better than random.
Training length	1 pass, 414 updates	Validation AUC was still rising (0.711 → 0.720 in the last quarter).
LoRA rank	16	Not tuned. A higher rank adds parameters, which only helps if there is enough data to train them.
Learning rates	2e-4 (LoRA), 1e-3 (heads)	Standard values; not tuned because of the compute budget.
Label noise	$\sigma = 0.5$ on the logit	Sets the ceiling: even the true chances only reach AUC 0.867.
Take-up rate	12.9% of tested customers	Few positives (653 in test) make AUC estimates noisy (±0.023).
Targeted offers	30%	Creates selection bias; makes take-up higher among offered customers.
Profile features	18 numeric + 2 categories, 90-day totals	Already capture much of the planted signal, which is why LightGBM is hard to beat here.
Test size	5,069 customers	Too small to measure gains under about 2%. About 55,000 needed for a 1.25% gain.
Compute	2×T4, 12-hour sessions	Forced LoRA, fp16, checkpointing, batch 4, one pass; round 2 used 6.4 hours.
Precision	fp16 with loss scaling	Halves memory and speeds up maths on the T4; loss scaling prevents lost gradients.